China-pub.com
下载

第3章　数据库和应用服务器

数据库是应用服务器的一个重要伙伴。实际上，应用服务器常被定义为 Web服务器和数据库之间的中介软件。

如果从数据库的角度来看应用服务器，你会发现这些术语都很熟悉；如果从 Web服务器的角度来看应用服务器，可能发现这些术语是全新的。

本章的各节涉及以下内容：

- 当今数据库的发展现状。
- 关系模型，以及。
- SQL。

3.1　数据库：发展现状

每个人都知道什么是数据库，但是，实际上“每个人都知道”这种说法并不那么明确。为避免混乱，必须知道如何谈论数据库，这很重要。在你拨开迷雾之后，下一个重要的任务就是理解目前正在使用哪些类型的数据库。然后，你可能需要了解一些新的术语（例如数据仓库和数据挖掘），才能跟上技术的进步。

人们很容易忘记在技术进步的道路上曾经有遇许多岔路，而且有些没有走下去的岔路已经是杂草丛生了。由于应用服务器常常在遗留系统世界里实现，你可能发现，自己要应付的数据库技术不仅是陈旧的，而且产生它的技术世界也已经不再存在了。

下面是对当前数据库世界的简要概述。

3.1.1　术语

“数据库”这个词是一个频繁使用的术语，而且在使用时充满了二义性——即使一个人使用也是如此。它可以指一个特定的项目（如订户数据库），数据的一个物理集合（存储在一个或多个文件中），或者是一个能实现这一切的软件产品（Oracle，DB2，FileMaker Pro，等等）。尽管通常通过上下文就可以明确它的含义，我们在本书还是选择中分别使用下列术语：

- 数据库项目　是指使用数据库技术实现的项目。它们由程序员、技术顾问、或者最终用户实现。在大型机环境里，常称它们为应用——例如工资表应用，记账应用，等等。在个人计算机世界里，常称作方案。在两个世界里，还可以称它们为系统。
- 数据库　是指数据的物理或逻辑集合，它们存储在计算机上，并通过软件来访问。在实现每个数据库项目时要使用一个或多个数据库。一个数据库可以用在多个数据库项目中。（例如，一个客户数据库既可以是计费数据库项目的一部分，也可以是市场支持项目中的一部分。）
- 数据库管理系统（DBMS）　是指能实现这些功能的软件产品。它们由软件公司开发，例

China-pub.com 下载

如Oracle，Microsoft，以及IBM。

3.1.2 目前使用的数据库管理系统

目前使用的数据库管理系统范围很广，包括要求信息技术人员提供全程支持的大型机产品，以及经过裁减适用于桌面计算机的产品。市场的另一个分支则是嵌入式数据库产品，它们构成每种产品的一部分，从手持设备，到个人计算机上的个人记账软件，以及 Web站点的支撑系统。

最后这个分支的重要性正在不断增大：随着这些年来个人计算机软件工业的不断变化和合并，许多公司都发现自己正在开发和支持的产品都是最初由其他公司设计的。这种异构的数据存储系统在许多情况下都会带来令人头痛的维护成本。

1. 数据库变得越来越大

数据库的规模正在迅速增大。例如记录交易事务的数据库，一旦把它们设置好，而且自动数据收集机制也已到位，它们就开始不断地增长。

在过去，通常需要对旧数据存档；继续保留这些数据不仅浪费磁盘空间，而且常常降低了DBMS的性能。在今天，这已不成问题了。

(1) 存储器变得越来越便宜

计算机存储器的成本正在下降（这和所有计算机的成本一样）。成本的下降反映在两个方面：

1) 现在你可以用比一年前更少的钱来购买一个硬盘。

2) 现在你可以用和一年前相同的钱来购买更大的硬盘。

实际上，磁盘存储器价格下降的并没有你所预期的那么多。相反，第 2点才是实际出现的情况：现在你购买的计算机所拥有的硬盘容量以 10亿字节（GB）为单位衡量；而在一年或两年前购买的计算机所拥有的硬盘却以兆字节（ MB）为单位衡量。

计算机存储器价格下降的结果，是一般用户（或企业）拥有了更多的可用空间。老一些的技术人员（来自几年前）可能记得，为了从一个项目转移到另一个项目，他们必须花费几个小时的时间进行存档和恢复。而今天，许多人根本用不完他们个人计算机上的磁盘空间。

此外，像RAID（廉价磁盘冗余阵列）这样的技术可以提供更高的性能和可靠性，在几年前这还是闻所未闻的事情。在十年前，磁盘镜像（即是否在重复的磁盘上存储数据的重复拷贝）的概念只应用在股市交易及其他事务处理系统中。今天，它已经是普通桌面用户的一个选项。

(2) 计算机变得越来越强大

当然，如果在处理数据时要耗费相当长的时间，那么存储大容量数据的能力也就变得毫无意义了。计算机硬件技术的进步使访问数据比以前快得多，因此大大减少了超大规模数据库所面临的问题。

不仅处理器的速度比以前更快，越来越多的计算机及其操作系统正在开始支持对称多处理（SMP），即几个处理器芯片在共享其他资源（如内存和磁盘）的同时，一起分担完成某项任务。当你把几个芯片捆绑到一起运行时，单个芯片的性能障碍就可以轻松突破了。

2. 数据库变得越来越小

在企业数据库变得越来越大的同时，手持设备可以访问的数据（甚至可以完全在手持设备

China-pub.com
下载

上实现的数据）正开始受到青睐。这些数据库通常需要和企业数据库进行同步，例如一个读表员的手持设备要把它的数据合并到城镇的水表数据库中。

3. 数据比以前多样化

数据已不再只是数字和字符数据了。现在的数据包括图形、图像及声音。与数字数据相比，这些数据的数据量通常相当大。此外，在检索图形、图像及声音数据时，用来操纵这些数据的工具常常要包括搜索机制。于是，有时候需要在数据库中存放一些类似于数据库的数据。

4. 数据需要更多人访问

最初，主要把数据库定位为帮助程序员实现项目的工具。（“你再也不需要编写输入/输出例程了。”）在不同项目中，通过数据库共享数据逐渐成为大家公认的，但并不总是能达到的一个目标。部分原因是因为每个项目都是独立设计的。如果对整个企业的数据策略进行统一规划，在数据完成时又常常过时了。到了20世纪80年代后期，经常可以见到公司拥有不兼容的系统和数据库；冗余和不一致的数据已成为普遍现象。

在一家银行中成立了一个特别工作组，目标是找出到底有多少个客户数据库存在。结果发现只有13个不同的名字-地址数据库。特别工作组的成员感动吃惊，大部分人都猜测可能有20～50个。

5. 数据库的合并和通信

最近十年来发展的技术允许数据库共享和组合它们的数据。这可以通过许多方法实现，但最终结果都是一家厂商数据库（例如IBM的DB2）中的数据可以和另一家厂商数据库（如微软的SQL Server）中的数据一起使用，即使这两个数据库位于两个不同类型的计算机上（例如一台IBM大型机和一台Sun工作站）。

许多机构常常允许由硬件和软件厂商来保存和管理它们的数据。在今天，更多的情况是机构收回了数据的管理权，并且要求随时都可以访问这些数据，而不必关心使用哪些硬件和软件来收集和存储数据。

6. 数据仓库

正在采用的一种方式是使用新出现的数据仓库——可以帮助机构把各种数据组合并存储到一个单一企业资源中的硬件、软件及相关技术。数据仓库通常是业务或事务数据的拷贝（复制）。如同以前为销售传票保留的复写副本。

数据仓库中的数据，要周期性地用支持业务和事务处理的联机数据库中的数据来更新。这些周期更新（常常是几小时一次）是修改数据仓库数据的唯一方式。因此，数据仓库的许多用户把它们看成是只读数据资源。

数据仓库通常包含细节数据——例如细到发货单一级的信息。概要数据常用于跟踪一家机构的业务；然而，通过访问细节数据，可以挖掘出一些隐藏的平均趋势和其他总结性信息。

数据仓库常用于联机分析处理（OLAP）。这是一些数据库加上专用的软件，为用户提供即时查询的功能。数据可以来自不同的资源；从某个机构的角度来看，数据仓库中的数据都是可访问的，就好像它们的格式和结构已经定义好了。换句话说，那些可以支持特定应用的特殊数据结构通常不放到数据仓库中——尽管数据本身存储在那里。这一点很重要，不仅因为数据仓库主要面向的是要做分析的最终用户，而且还因为它在设计时没有考虑事务系统所要求的更新

China-pub.com 下载

性能。

7. 数据挖掘

进行联机分析处理的方法是使用数据挖掘 —— 这种技术允许用户在一个联机环境中快速和高效地分析数据仓库中的原始数据。通常数据仓库中的数据要和其他外部数据组合在一起完成分析（例如，市场应用中就特别需要人口统计数据）。数据的结构允许对来自不同数据库的数据进行操作，就好像它们包含了相似的底层数据（而实际并非如此）。

数据挖掘和数据仓库对某些机构的影响怎么强调都不过分。能够迅速访问到原始数据一直是统计人员的梦想。在传统的统计分析中，数据经常相当简单，只能反映各种观测数据的平均值；统计分析却常常非常复杂。使用这些工具，数据本身变得相当复杂 —— 在许多情况下，几乎不能做任何概括。因此，统计分析就常常相当简单的了：不需要做任何的推算。

8. 数据库的秘密

最后，需要注意的重要一点是，数据库是目前最成熟的计算机软件产品之一。这就是说，尽管它们不像最新的 Internet技术那样耀眼，但它们常常是最可靠且质量最好的产品之一。在 20 世纪60、70、80及90年代暴露的缺陷都已经解决了。此外，数据库经历了这一切 —— 从第一次在分时大型机上的尝试，到最近试验想在无线手持设备上实现的小片数据库。

可靠性、同步、并发、事务处理、回滚、文档以及安全，这些都是近几十年来谈论的话题（包括试验和实现）。数据库已尝试了这些技术，而且已经完全实现了。

除此之外，主要的数据库产品之间都具有可比的功能和价格。很自然，每一家主要的数据库厂商都拥有自己的应用服务器产品。它们的大用户通常不想更换数据库，但每个客户都希望能应用所有的最新技术。

9. 桌面上的数据库

基于个人计算机的数据库产品包括 FoxBase/dBase、FileMaker Pro、Microsoft Access以及其他一系列产品。 FoxBase/dBase是最古老的产品之一，实际上现在已很少卖了。但是，使用这些产品的数据库仍然在广泛使用。

在桌面市场上，FileMake Pro仍然是一个主要产品；Microsoft Access是一个过渡产品，因为微软鼓励用户考虑转向 SQL Server。

3.1.3 这对应用服务器意味着什么

应用服务器在Web服务器的界面和数据库中的数据之间提供了一个连接。这些数据库可以是联机的（如面向事务处理的数据库，在发生事件时不断更新），也可以是静态的数据仓库，它们包含的是已完成事务的拷贝。

1. 让不可能变成可能

那些程序员用作工具、用来简化数据读写操作的数据库，已经被当作企业资产的数据库取代，它的数据不仅要用于事务处理，而且要用于分析。从程序员和信息技术人员的角度看，企业的最大目标是对平台和数据库系统进行标准化；但从管理角度看，主要的目标更应该是实现某种类型的开放功能，像第2章中示意图所描述和展示的那样。

这就意味着公司的管理者只画出从一个数据库到另一个数据库，或者从一个系统到另一个

China-pub.com
下载

系统的流程图，而不管这些产品和平台实际上是否能彼此通信。管理者认为这不是他们的问题但是，他们需要这样。应用服务器就非常适合于使这种结构成为现实。

2. 使企业信息环境规则化

越来越多的机构都在检查讨论过的各种体系结构上的信息资源。公共数据库（至少在企业内部）可以可用于各种查询和更新 —— 这些访问要么是直接的，要么通过它们的数据仓库拷贝进行。采用这种体系结构不同于选择特定的硬件或软件产品：它是一个演变了几十年的体系结构，而且不会有什么风险。

如果你的机构使用其他的体系结构（或者还没有任何体系结构，例如像 Topsy那样，是刚刚建起来的临时环境），现在就可以考虑向这种类型的体系结构转移。由于这种体系结构实际上并不是一个革命性的改变，因此，这个转变过程可能不会比重新命名你的模块和系统花费更多的工作量。在某些情况下，它意味着要规划清除非数据库系统；通常这些都是遗留系统，并且拥有古老的数据库（或者定制编写的输入 /输出例程）。

3.1.4 其他类型的数据库

和任何技术一样，数据库也有一些错误的起点和歧途。在某一时刻，并不总能明确什么才是主流方向：你所拥有的遗留系统可能采用的就是现在已不再使用的技术。

1. 数据库机

在20世纪80年代后期，人们开发了特殊用途的数据库计算机，而且有的确实已进入了市场。它们的硬件和操作系统专门对数据库需要的操作类型作了优化（例如大数据量输入 /输出，索引操作，等等）。

随着更加强大的计算机的出现，这种专用的技术就不那么重要了。现在的数据库机通常是一些标准计算机，通过管理而不是硬件配置达到专门的目的。

2. 数据字典和数据模型

在20世纪80年代中期，盛行的是企业数据字典和数据模型。它是指开发一个企业范围的数据管理模式，把机构中的所有业务都包容在里面。在最极端的要求下，必须编写（或重写）机构中的所有软件。在松一些的要求下，它描述了一个新系统必须符合指南。

不幸的是，经常在建立机构的数据字典和数据模型的期间里，机构本身已经改变了 —— 通过兼并或收购其他公司、一般的商业开发、或者其他正常的事件。这种数据管理的集中模式导致许多项目预算超支或出现大量误解。

如果你把这种集中规划方式和现在使用的数据仓库方式作一比较，就会注意到数据仓库允许一个机构采用对他们来说最合适的办法来实现和运转自己的系统。然后，在把数据存储到数据仓库的时候再对数据进行拷贝和必要的转换。这种方式尽可能地结合了集中化和局部控制的优点。

3. 非关系模型

几乎所有目前使用的数据库都是关系型的。其他类型的数据库也存在（而且一直都存在）。在非关系数据库中，最重要的三种非关系数据库是倒排表、层次数据库和网状数据库。

倒排表是从平坦文件发展来的产物。平坦文件中各字段的内容被拷贝到另一个单独的文件

China-pub.com 下载

中；这样，你就可以从一个倒排文件中读到给定字段的所有值。你可以先定位自己感兴趣的字段值，然后再读取这些记录。按索引顺序访问方式（ISAM）文件是倒排表数据库的一个例子。

倒排表常用于对文本信息进行索引；在今天，这种倒排表又被搜索引擎用来检索Web页面。在这种情况下，没有显式的字段—只有一个长的文本字符串。先从文本中抽取有意义的单词（除了“and”、“whereas”等之外的单词）；然后为每一个单词创建或更新一个记录，把这个单词及其定位的URL放在一起。通过这种方式，可以为“数据库”创建一个记录，其中包含具有这个单词的Web页面的URL。类似地，也可以为“invert”创建一个记录（英文单词都去掉变形后缀，因此“inverted”也存储在“invert”中）。为了找到关于倒排表数据库的所有URL，需要比较invert、list、database的记录，并显示出三者都包含的公共URL。

层次数据库要比倒排表数据库更加结构化一些。这里，把数据从逻辑上分组。你可以拥有一个叫做Department（部门）的数据元素；在这个元素的内部，又可以有一个Manager（经理）元素。在Manager元素内可以定义电话号码。IBM的IMS就是一个层次数据库的实例。

层次数据库的问题是它们很快就变得复杂和冗余。一个部门经理也可以是另一个经理手下的员工——而数据库却没有实际世界那么灵活。

网状数据库试图解决层次数据库的灵活性，它允许通过许多方式在节点之间建立关联。这个问题可以很好地解决灵活性，但它的复杂性很快就超过了大多数人所能掌握的范围。网络数据库的一个例子是Cullinane公司的IDMS。

值得注意的是，层次数据库和网状数据库很快就变得相当复杂，以至于许多机构都对数据库中可以采用的级别个数作了强制限制。为满足共享数据的需要，必须数据把按用逻辑方式进行组织，并允许不同人使用不同的工具来访问。

3.2 关系模型

数据库设计的关系模型非常简单。数据被放置在表中，看上去像电子表格的二维矩阵。每一列代表一类数据，如地址、ID号、年龄等等。每一行代表一个观察点，如一个人、一个库存项目、或者一个URL。

关系数据库允许连接表（或表的一部分），来构造非常复杂的数据集合（这些数据集合可以达到与任何层次或网状数据库相同的复杂度）。关系数据库与它们的区别是这些复杂数据是动态构造的——在需要时才创建。数据本身存储在简单的二维表格中。

不仅数据存放在表中，而且在一个关系数据库中，数据库操作的结果本身也是表。这些新的表可能是来自一个或多个表的数据集合，或者是来自一个或多个表的数据的子集；不仅如此，在操作数据库的每一个步骤中，所处理的都是表格。

这些中间表可能是临时的，和存储在磁盘上的“实际”表格不同。从一个程序员或最终用户的角度来看，一个表是实际的还是临时的无关紧要。

除了这个基本原则（所有事物都是表格）之外，你还需要了解有关关系数据库的一些其他知识。下面几节将向你介绍有关术语和重要的规范化问题。

China-pub.com
下载

3.2.1 术语

在关系模型中使用的术语都是简单的英文单词。最基本的术语都是成对出现的：其中一个元素是一个常用单词（有二义性）；另一个比较精确，而且建立在数学集合论的基础上。

正如前面提到的，关系数据库的所有数据都存放在表中。一个表也被称作一个关系。

注意，数据的物理存储由数据库管理系统负责。一个关系数据库向用户提供数据，就好像数据存放在表格中，而不管实际情况是否如此。

表的每一行包含一个实体（一个人、一个库存项目、等等）的数据。一行也被称作一个元组（tuple）。

表中的每一列包含的是一个字段（field）或一个属性的数据。对于表中的每一行（如人、库存项目、Web站点等），一列可以表示它的年龄、ID号、颜色或者地址。

主键是一个字段，这个字段包含的数据对表中每一行来说是唯一的。一个 ID号可以是一个主键。名字、地址、颜色以及其他类似属性不一定是唯一的，因此通常不能用作主键。关系数据库允许你坚持主键的唯一性，这样人们就不能向主键加入重复的名字或其他的值。

域（domain）是指一个属性（或字段、列）的合法取值空间。和主键一样，数据库软件一般允许你坚持合法取值空间，这样你就不会错误地为一个颜色属性输入“ dog ”值。通常你需要编写一些附加的特殊软件，以坚持这些规则。

一个表的基数（cardinality）是指该表包含的行的个数；而度数（ degree）是指包含的列的个数。

当把数据转移到数据仓库中时，它常常需要重新结构化和重新组织。这样，即使数据可能来自一系列不同的数据库，数据仓库也可以收集逻辑上相关的数据。数据仓库使用术语“ 维数 ”表示这样一个逻辑结构。维数可以是像时间、地点这样的概念；它们收集诸如月份、日期、年或者城市、州或省、以及国家这样的数据库字段。在其他情况下，一个多维数据库的维数实际上只是一个单独的二维数据库。

3.2.2 规范化

关系数据库的强大功能主要来自它的核心概念（表）的简易性，以及可以执行的特写的组合与抽取能力。为了使基本表格能够保持简单，以利于对它们操作，需要把关系数据库的表格规范化。规范化（normalization）意味着数据的结构比没有规范化更简单。（这里使用的“ 简单 ”是严格数学意义上的术语。结构可以相当复杂，但不能找到比它更简单的结构。）

规范化源自逻辑学、集合论和数学。它看上去非常清晰和雅致。不幸的是，在实际世界里，规范化常常更多的是一门艺术而不是科学。

首先，需要理解数据库中数据的含义，这不只是写下字段名那么简单。举一个简单的例子，了解颜色能否作为唯一的主键是一件重要的事。对于孩子们的积木来说，它可能无法作为一块积木的唯一标识符；然而，在一本绘画示例书中，颜色可以作为一个唯一的主键。

其次，在一个共享数据库的例子中，数据的含义可能会对不同的用户有微妙的变化。如果你觉得这不可能，请考虑一下人们对单词含义有过的争论。规范化之所以有意义，是因为人们

China-pub.com 下载

在组织数据时有规则可循。你可能需要规范化自己的数据，这时你不仅需要知道这些规则是什么，而且需要了解哪些数据因素和这些规则相关。

一共有五个范式，它们一个比一个复杂。第一范式是为关系数据库规定的：如果它们不符合这个范式，它们也就不是关系数据库了。大部分关系数据库都用到了前三个范式。第四范式和第五范式只在某些情况下使用。（并非所有的关系数据库教科书都提到了第四和第五范式；它们是在20世纪70年代后期才提出来的，而关系模型及其前三个范式可以追溯到 1970年。）

1. 第一范式：消除重复的组

这是关系数据库的基本规则。考虑一个表，它为一个拥有若干仓库的公司存储库存信息。典型的数据记录可能类似于表 3-1所示的结构。

表3-1 含有重复组的库存表

库 存 表	库 存 表
零部件ID	仓库2的存货量
仓库1的名字	仓库2的订货量
仓库1的存货量	仓库3的名字
仓库1的订货量	仓库3的存货量
仓库2的名字	仓库3的订货量

在这个结构中有三个重复组。每一个仓库的名字、存货量、订货量都对应一个字段。这违反了第一范式，至少会带来三个潜在的问题：

1) 每个记录最多只能存放三个仓库的信息。没有任何零件存放在多于三个仓库的情况下，这种结构才能正常工作。如果某个零件只存放在一个或两个仓库中，就会浪费存储空间。

2) 难以高效地搜索。如果你想查找一个特定仓库的库存信息，你不得不读取每一个记录，并查看该仓库是否在任意三个字段中存在。

3) 取消这些限制会给数据库的设计者和用户带来可怕的恶梦。如果一个零件需要在多三个以上的仓库中存放，你只能为这个零件创建重复的记录（可能要在零件号的后面加上一个＃ 1或＃2）。

什么是重复的字段？表3-2展示了一种解决上述数据库结构部分限制的方案。有些人可能会认为这个表结构不再拥有重复组了 —— 毕竟，每个字段已指定为一个唯一的仓库。

表3-2 含有隐含重复组的库存表

库 存 表	库 存 表
零件ID	Hudson仓库的订货量
Valatie仓库的存货量	Ghent仓库的存货量
Valatie仓库的订货量	Ghent仓库的订货量
Hudson仓库的存货量	

但是，这里仍然存在问题：

1) 它把仓库的结构作为表的一部分。如果公司关闭了一个仓库，或者新建一个仓库，则所有的表记录都需要修改。

2) 它仍然难以搜索。如果你想计算 Ghent仓库的全部零件库存量，你必须读取所有的记录。

China-pub.com
下载

此外，如果你想查找存有一个给定零件的任意仓库，你也必须搜索所有记录。

这就引发了一个在数据库设计中常见的问题：一个人认为它是重复字段，而另一个人可能觉得它是一个唯一的标识符。这就是为什么数据库设计需要判断和决策的原因；在使用自动设计工具时通常需要做一些调整。

规范化的表结构　这个表的正确结构应当是使用两个相关的表，如表 3-3所示。

表3-3　规范化的库存表

仓　库　表	库　存　表
仓库ID	仓库ID
仓库名	零件ID
	存货量
	订货量

仓库表包含仓库的标识符（ ID）及其名字。在现实世界里，它还要包含一系列其他字段，例如地址、电话号码等。库存表包含有关在一个特定仓库中存放的一个特定零件的信息。要想找到仓库的名字，你需要查看仓库表（使用同时存储在两个表中的仓库 ID）。

库存表中的记录只有在需要时才会存在：如果一个仓库没有存放任何零件，那么它只在仓库表中有一个记录，在库存表中不会有任何零件记录。类似地，对于一个存放在多个仓库中的零件，可以轻易地把它的信息存储到库存表的记录中，而且最多允许 3个（或10个，或100个）仓库。

现在，你可能想知道是否在数据库结构的简易性和实际性能之间存在折衷。毕竟这个结构在获取完全的库存信息（包括仓库名）时需要访问多个表格。前面所展示的表允许把一个零件的所有信息存放在一条记录中，因此只用一次磁盘访问就可以检索出来。

现在的回答是，存储空间变得越来越便宜，数据访问速度也越来越快。而越来越紧缺和昂贵的是程序员的时间。特别是对使用先进缓存算法使磁盘访问最小化的现代数据库来说，理论上，规范化数据库所要求的多次访问带来的性能下降完全可以忽略不计。

2. 第二范式：消除冗余数据

如果你担心性能，可能会诱使你在库存表中把仓库名和仓库 ID存放在一起。毕竟这只是很少几个字符，而且在你生成报表时节省了一次额外的磁盘访问。

不要这样做：它违反了第二范式 —— 消除重复数据。这条规则有充分的理由存在。简单地说，数据冗余会带来很大的由于不正确而产生的风险。如果把仓库名存放在库存表中，那么每一个零件记录都拥有对应的仓库名。如果要改变仓库名，你该怎么办？修改成千上万个库存表记录？如果使用无实际含义的仓库 ID来连接库存记录和仓库记录，你就可以安全地改变仓库的名字（只需在仓库表中修改一个记录），这样每个库存表记录对应的仓库名都将是正确的。

有经验的数据库设计者都非常喜欢无实际含义的标识符。一旦你在一个标识符中加入了其他信息（例如名字），那么你就违反了第二范式（至少在理论上是如此）。

什么时候数据是冗余的　和第一范式一样，在涉及到具体数据时，往往存在许多不同的解释。冗余数据可能很明显（比如仓库的名字），但也可能更深奥。

China-pub.com 下载

考虑一个存储发票信息表的情况。你可能还要有一个相关的表，用来存放发票上每个物品编号的名字和价格。通过在发票中放置物品编号，就可以在需要的时候从第二个表中检索辅助的信息。

但是，如果这是一个账单发票怎么办？如果你需要存储在完成销售时实际收取的价格怎么办？（你的第二个表可能包含的是当前价格。）在这种情况下，更合理的办法是把价格放到发票表中 —— 尽管刚开始它看上去是冗余的。随着时间的推移和价格的变化，就会发现这个数据实际上并不是冗余的。它将是不一致的 —— 也应当如此。发票反映的应该是销售时的价格，而不是现在的价格。

相反地，你可能需要把价格从物品表中取出，单独建立一个物品目录表，这样就杜绝了出现差异的可能性。要记住，数据库设计要求你理解数据的含义以及数据将如何使用。

如何处理冗余数据　有两个基本方法来对付冗余数据。你可以把冗余数据拉出来，放到一个单独的表中；或者，你也可以用某种方式明确数据实际上并不冗余。就像前面提到的价格随时间变化的例子，你可以把物品表中的字段称作“当前物品价格”，而把发票表中的字段称作“交易价格”，这样就明确了你实际处理的是两个不同的数据项。

如果你去查看现存的数据库，就会不断地发现冗余数据；为某个系统创建的一个完全规范化的数据库所存放的数据可能和另一个数据库重复，而这个数据库是由同一个机构中另一个地点的其他人建立的。下决心要求人们修改正在使用的数据库结构并不是最好的处理方式。这个问题的唯一解决方案是理解数据的含义以及需要怎样去处理数据。（这是数据仓库所面临的一个常见的挑战：它们常常要处理实际上不一致但在逻辑上又是等价的数据。）

3. 第三范式：消除与主键无关的字段

这个问题的一个经典例子是地址数据库，它包含邮政编码和城镇的名称。你可以使用这样一个地址数据库查找某个给定城镇的邮政编码，或者相反。但是，这个数据库只能查找那些出现在它所包含的地址中的邮政编码和城镇。

解决方案还是把数据分割到单独的表中。在极端的情况下，你只在地址数据库中存放邮政编码或城镇名 —— 由一个单独的数据库提供全部的城镇名和邮政编码。这就再一次涉及到决策问题：对于像邮政编码和城镇名这样的小数据项，违反第三范式并不见得是一个可怕的错误。但是，要知道，人们可能要依靠你这个非规范化的表来查询邮政编码和城镇——而且他们可能会找不到。

3.3 SQL

SQL最开始只是一个缩写，现在已经是一个专用名称了。它的目的是操纵 SQL-数据——它可以处理的关系数据的子集。（某些类型的关系数据不能用 SQL操作，但是对于本书讨论的应用类型来说，它们并不常见。）

关于SQL要记住的最关键的一点是：它是描述性的，而不是过程性的。这一节将概述这句话的含义和意义，并简要介绍SQL的语法。SQL语法并不复杂，而且有许多资源可以参考，包括书籍和联机参考文献。这一节给出的只是主要语法的基本概述。

China-pub.com
下载

3.3.1 过程性和描述性程序设计

SQL最重要的特点是：它是描述性的而不是过程性的。传统的程序设计（就像办公室、工厂以及其他环境中的手工过程一样）主要是过程性的。在这种过程性的方式下，操作过程通常是这样的：

- 获取的第一个是……（在装配线上、在新邮件中、等等）对象。
- 对它进行测试，以确定对它做什么。
- 完成所有必需的操作。
- 获取下一个对象（从装配线上、在新邮件中、等等），并重复上述步骤，直到没有其他对象为止。

对于描述性程序设计，其结构更加简单：

- 获取所的……（在装配线上、在新邮件中、等等）且需要特别处理的对象。（这是上一个列表中的前两个步骤。）
- 对每个对象完成必需的操作。

过程性方法中的测试步骤（确定对每个对象做什么）被集成到第一个步骤中：获取所有需要进行特别操作的对象。这就使处理工作变得更简单，因为不再需要对每个对象进行内部测试。

> 注意，这类以“获取所有的……的对象”开头的程序设计最终可能没有得到任何记录或只得到一个记录。实际上，在逻辑上你可能知道只有一个记录会满足“获取所有的……的对象”的请求。不管怎样，即使你处理的是一个单独的记录，描述性程序的结构仍然和上面描述的结构一样。

结构化程序设计（在20世纪70年代开发）抛弃了“意大利面条式代码”——即含有 goto语句的计算机代码，它使程序的执行跳来跳去，以至于人们难以读懂程序。描述性程序设计又前进了一步，它抛弃了条件语句（因为它们已被集成到开始的语句中，并获得所有具有特定特征的对象）。结果是现在编写的代码在从头到尾的执行过程中没有跳转和条件语句。这样的代码更有可读性，也更容易测试：因为只有一个执行序列。

如果你理解了描述性程序设计的精髓，就会发现自己长期积累的一些编程观念会发生改变。在过程性程序中，你常常需要为操作的数据存储大量的临时性信息。你可能需要一个变量，以指定生成的报表是总结性的还是细节性的；你可能要有一些标志，告诉你是在一个月的月初还是月末时整理账目；或者，你可能需要一些指示器，向用户警告可能的错误。

在过程性程序设计中，这些信息都存放在数据库表格中。纯粹的过程性程序设计除了 SQL语句之外几乎不包含其他东西；而且， SQL也不提供变量。因此，在任意给定的时刻，不仅数据由数据库表格反映，而且在工作时可能需要的有关数据的临时信息也由数据库表格反映。旧时代的程序员可能会抱怨低效性，因为要获取这些附加信息必须进行磁盘访问；但是，正如前面所提到的，现代数据库已使用缓存解决了性能问题。

这种结构把事务以及其他操作的状态存放在数据库中，而不是放在程序的内存中，这一点正好和无状态的HTTP协议高度兼容。如果你试图实现一种基于 Web的系统，而且不只是让它简单地显示准备好的HTML页面，你就会知道要想很容易访问这样的数据是一件多么困难的事情。

China-pub.com 下载

3.3.2 基本SQL语法

SQL可以用在使用其他编程语言编写的程序中，例如 COBOL、C、C++、Visual Basic、或 Java。通常不单独使用SQL编写程序。

1. 选取

无论你是通过文本方式键入查询语句，还是使用一个图形化用户界面进行设计，通常都在创建SQL查询。（即使在没有SQL查询的情况下，数据库的设计者也通常用 SQL术语来考虑问题，他们总是有意识或无意识地把他们的查询语言映射为 SQL。）

最基本的SQL语句是Select（选取）语句；它对一个或多个表操作，并创建结果（一个新的表）。尽管Select语句可以非常复杂，它的基本形式却很简单：

```
Select column(s) FROM table(s)
<WHERE condition>
<GROUP BY condition
    <HAVING grouped characteristic>>
```

子查询　因为Select语句的结果还是一个表，你可以在 Select语句的From从句中使用Select语句，如下所示：

```
Select Name, Age  From
    (Select Name, Age, Address From Customers Where Age > 18)
```

（尽管这个语句可以表示为一个单独的 Select语句，但许多复杂的语句确实得益于这种 Select语句的多层次结构。）

Select *　除了指定要检索的具体列之外，你还可以使用一个星号，要求选取表中的所有列，如下所示：

```
Select * from Customers
```

（这是一个可以构造出的最短的 Select语句。它返回表中的所有行和所有列。）

Where 条件　在Where从句中可以定义比较值（例如 Age > 18）或者集合操作（如IN）。你可以创建一个临时表（当然是使用一个 Select语句），并使用这个临时表来测试是否使用主 select语句返回的行。

Group By/Having　Group By从句允许你通过某些值（通常是一个计算值，例如总和或平均值）对返回的行分组。如果同时使用 Having从句，你还可以只在结果中包括那些来自具有特定特征组的元组（例如总和大于 X，平均值小于 Y，等等）。

Select语句示例　下面是一些典型的Select语句：

```
Select Names from Customers
```

这个语句从Customers表中选取所有的客户名。

```
Select Names from Customers Where Balance > 100.00
```

这个语句从Customers表中选取所有余额大于 100的客户的名字。

```
Select Names from Customers Where Balance > 100.00 ORDER BY Balance
```

这个查询与上一个相似，区别是结果将按照余额从小到大顺序排列。

China-pub.com
下载

可以同时对多个表查询；这时，需要用查询中的表名来限定列名，而且需要表示两个表中的记录之间的关系。在下面这个查询中，记录取自 Customers和Accounting两个表；这两个表通过一个共有的字段（叫做ID）进行匹配：

```
Select Names, Balances  from  Customers, Accounting
  Where  Customers.ID = Accounting.ID
```

其中，Customers.ID=Accounting.ID部分把两个表的记录连接起来（假定 Customers和Accounting表都有一个唯一的客户ID，而且它们的ID是相同的）。用一个公共字段从两个底层表创建一个新表的过程叫做连接（join）。

2. 游标

Cursors（游标）与一个表相关联，是作为某些数据库操作的结果而创建的（例如查找所有是……的记录）。游标可以定位在表中任意记录的上面、后面或者前面。当这样对它定位时，它对应的记录就可以用程序操作，然后游标自动前进到下一个记录。

大部分典型的数据库访问都通过打开的游标完成 —— 也就是说，通过一个查询构造一个与游标相关联的表。后续的访问（如检索每个记录）只需要少得多的数据库处理开销。

3. 插入

SQL的Insert（插入）命令向一个表中插入一行数据。数据通常必须经过适当的编辑（例如含有一个唯一的主键，某些字段必须是数字值）。你会经常用到Insert命令向数据库驱动的Web站点添加数据，尽管你使用的应用服务器通常会自动为你生成这个命令。

通常要插入到数据库中的数据都是通过一个 HTML表格从用户那里收集的。

4. 删除（Delete）

SQL的Delete（删除）命令删除一行由对应的唯一主键指定的数据。在删除记录时也需要适当的编辑，以防止你删除几个表格相互关联的记录的某个部分（引用完整性）。

和Insert一样，通常你不会见到这个代码 —— 你的应用服务器软件会自动生成它。

5. 更新

SQL的Update（更新）命令修改一个表中现有的数据。你可以使用 Update命令的变形来完成大批量的更新（例如把所有值都乘上一个常数）。Insert和Update的区别在于Insert创建一个新的记录，而Update修改一个已有的记录。

6. 创建

一系列的SQL Create（创建）语句让你创建关系数据库的对象。这些对象既包括表，也包括一些辅助结构，例如视图和索引。

视图（View） 视图可以看做是临时表。一个选取操作的结果总是一个表；这个表可以被存为一个视图。它并不像一个表那样实际存在，但你可以在引用一个视图时把它看做是一个表。每次你引用一个视图时，就重新执行创建这个视图的 Select语句，包括它的条件和准则。

索引（Index） 数据库软件使用手头拥有所有的信息来完成你的查询。它可以为数据库中的数据创建索引 —— 你也可以自己创建索引。如果你知道数据是怎样检索出来的，你就会知道它们是很有用的。

3.3.3 ODBC和JDBC

China-pub.com 下载

在20世纪90年代早期，微软发布了它的开放数据库互连技术（Open Database Connectivity，ODBC）。ODBC以SQL标准和其他机构所做的工作为基础（包括 X/Open的调用层接口和SQL访问小组1992年发表的计算机辅助工程规范—— SAG SQL CAE），它是一个与一组运行时间库配套的应用编程接口（API），允许你动态地访问各种不同的数据库。

你可以在你的程序中使用ODBC标准编写数据库调用。在运行期间，ODBC库连接到一个特定的数据库驱动程序，由它负责把标准的ODBC调用转换为实际的Oracle调用、DB2调用、SQL Server调用 —— 甚至是对非数据库的调用，如Excel电子表格或平坦文件。任何对象，包括字段名和数据库名，都可以在运行时改变。这就允许你编写一个抽象的数据库，它只在运行时才实际建立。

最近，人们又开发了一套与其相伴的API，称作Java数据库互连（JDBC），其目的是在Java环境中执行这些相同的操作。

3.4 小结

几乎所有目前使用的数据库都基于关系模型；它们使用SQL作为基本数据操纵语言。由于它的标准化，人们可以把来自不同厂商的数据库集成到一个标准的体系结构设计中 —— 比如使用应用服务器的体系结构。

除了Web服务器之外，应用服务器可以通过SQL和数据库紧密地结合。所有这一切都要归功于一系列的技术；其中最重要的是组件软件的兴起。这是下一章的主题。